"Año de la Esperanza y el Fortalecimiento de la Democracia"

**CARRERA: INGENIERÍA DE SISTEMAS COMPUTACIONALES**

# CURSO: TÉCNICAS DE PROGRAMACIÓN ORIENTADA A OBJETOS

## PRÁCTICA DE CAMPO: SEMANA 01 Y 02

**AUTORES:**

Alcántara Salazar, Walter Iván. Código: N00436696

Chávez Saldaña, Miguel Angel. Código: N00419003

Rojas Fernández, Karina Mishel. Código: N00443936

Romero Conde, Jhon Charles. Código: N00441923

Eli Thalia Ledesma Briceño. Código: N00310472

**DOCENTE:**

Martín Eduardo Torres Rodríguez

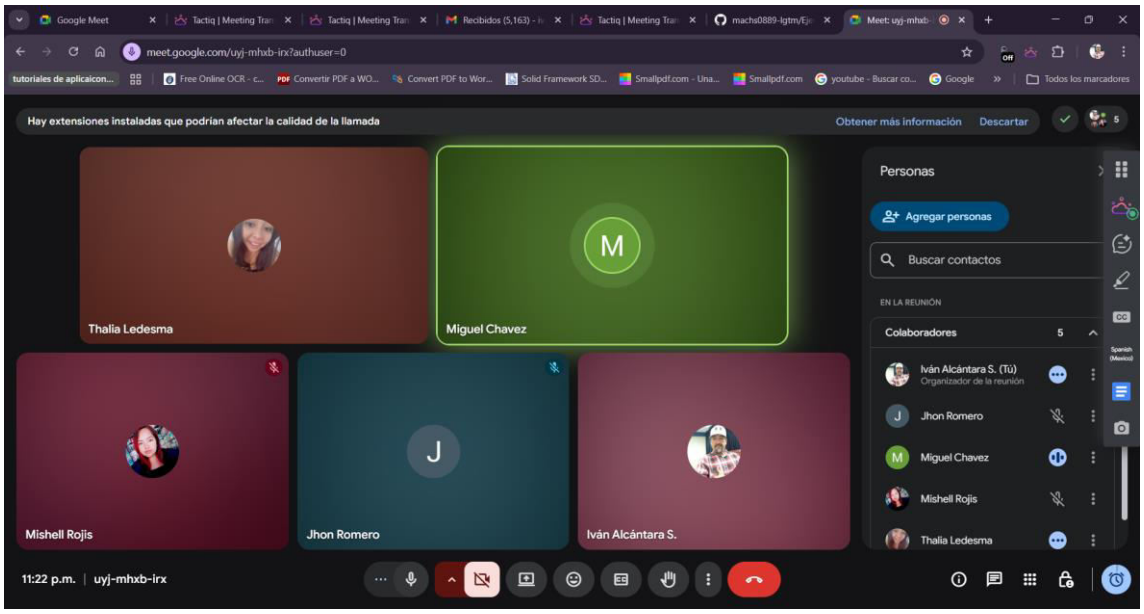
**NRC:**

10723

**GRUPO:**

08

**2026**




# REPOSITORIO DEL TRABAJO:

https://github.com/IvanHoeFX/Taller_Semana1_2.git

# TALLER PRÁCTICO – SEMANA 1 y 2

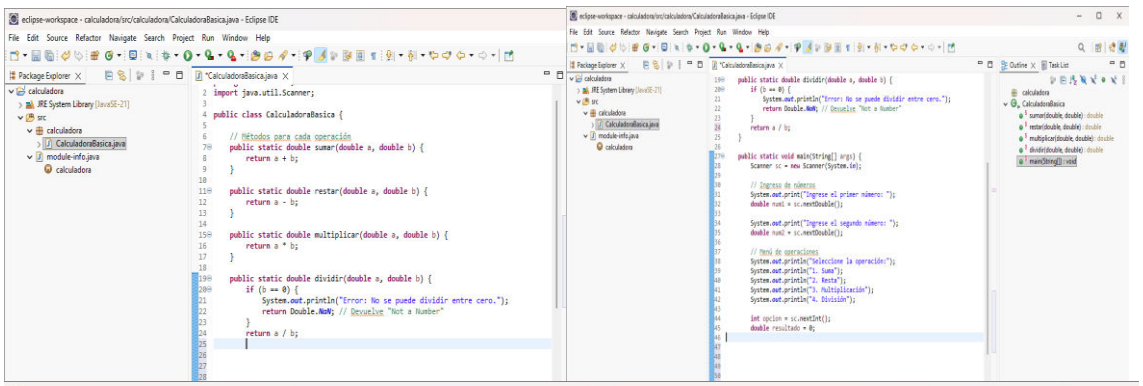
Curso: Técnicas de Programación Orientada a Objetos
Duración: 1 hora
Lenguaje: Java (Eclipse)

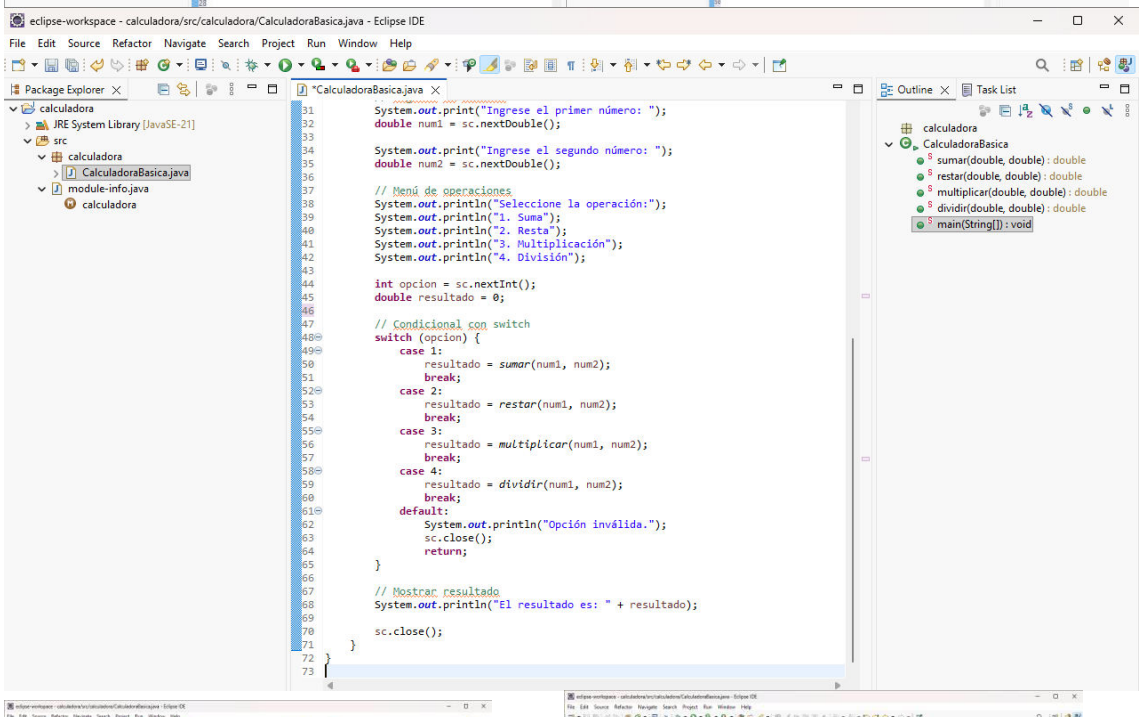
## CASO 1: Calculadora básica (Condicionales + métodos)

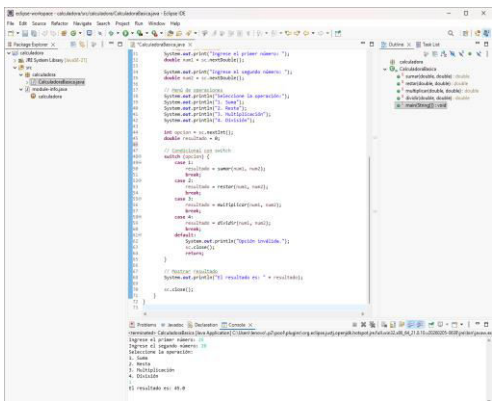
Consigna:
Desarrollar un programa que permita:
- Ingresar 2 números.
- Elegir una operación: suma, resta, multiplicación o división.
- Mostrar el resultado.

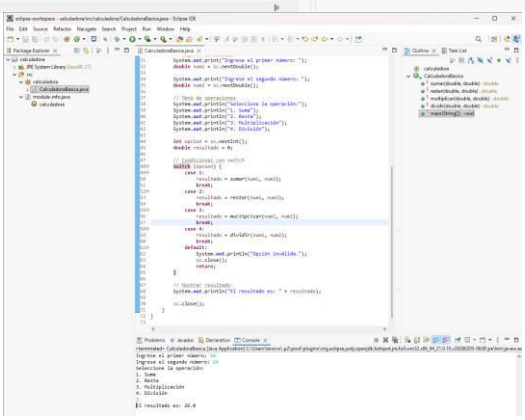
Temas:
- Scanner

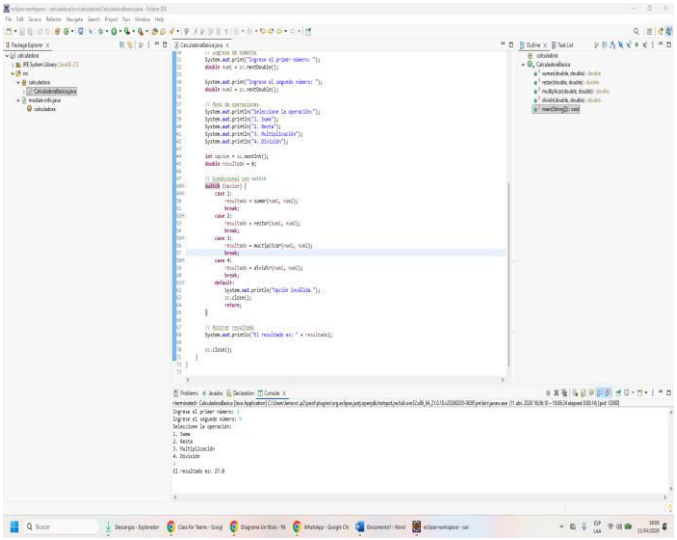
- Condicionales (if / switch)
- Métodos

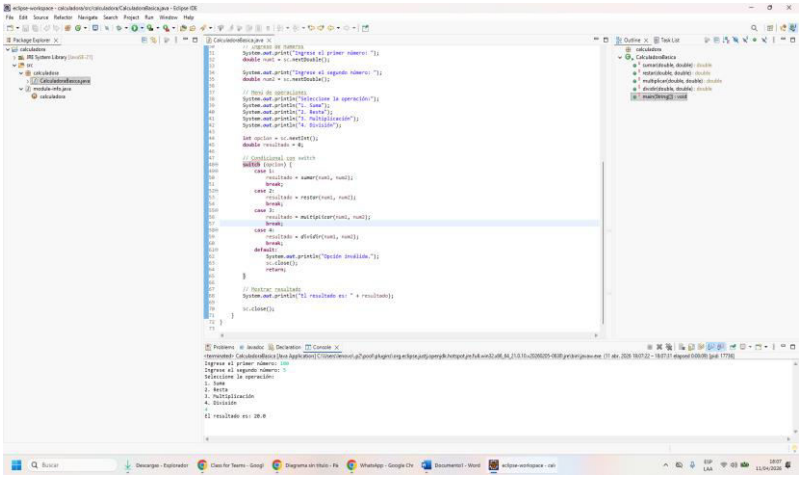










Código de programa básico según requerimiento:

```
package calculadora;
import java.util.Scanner;

public class CalculadoraBasica {

        // Métodos para cada operación
        public static double sumar(double a, double b) {
    return a + b;
        }

        public static double restar(double a, double b) {
    return a - b;
        }

        public static double multiplicar(double a, double b) {
    return a * b;
        }

        public static double dividir(double a, double b) {
    if (b == 0) {
       System.out.println("Error: No se puede dividir entre cero.");
```

```
        return Double.NaN; // Devuelve "Not a Number"
    }
    return a / b;
        }

        public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    // Ingreso de números
    System.out.print("Ingrese el primer número: ");
    double num1 = sc.nextDouble();

    System.out.print("Ingrese el segundo número: ");
    double num2 = sc.nextDouble();

    // Menú de operaciones
    System.out.println("Seleccione la operación:");
    System.out.println("1. Suma");
    System.out.println("2. Resta");
    System.out.println("3. Multiplicación");
    System.out.println("4. División");

    int opcion = sc.nextInt();
    double resultado = 0;

    // Condicional con switch
    switch (opcion) {
        case 1:
            resultado = sumar(num1, num2);
            break;
        case 2:
            resultado = restar(num1, num2);
            break;
        case 3:
            resultado = multiplicar(num1, num2);
            break;
        case 4:
            resultado = dividir(num1, num2);
            break;
        default:
            System.out.println("Opción inválida.");
            sc.close();
            return;
    }

    // Mostrar resultado
    System.out.println("El resultado es: " + resultado);

    sc.close();
        }
}
```

## CASO 2: Clase Persona (POO básico)

Consigna:
Crear una clase Persona con:
- Atributos: nombre, edad
- Método: mostrarDatos()

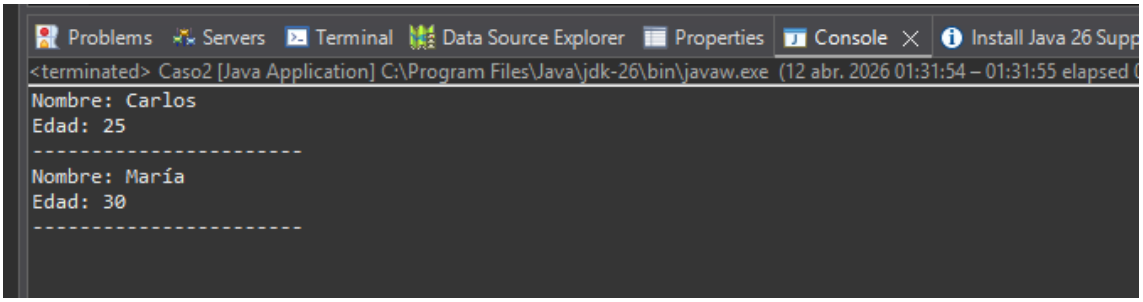
Desde main, crear 2 objetos y mostrar su información.

Temas:
- Clases y objetos
- Métodos
- Encapsulamiento básico

## DESARROLLO

Caso2.java

```java
public class Caso2 {

    // Clase Persona (definida dentro del mismo archivo)
    static class Persona {

        // Atributos
        private String nombre;
        private int edad;

        // Constructor
        public Persona(String nombre, int edad) {
            this.nombre = nombre;
            this.edad = edad;
        }

        // Método para mostrar datos
        public void mostrarDatos() {
        }

        // Getters y Setters
        public String getNombre() {
            return nombre;
        }

        public void setNombre(String nombre) {
            this.nombre = nombre;
        }

        public int getEdad() {
            return edad;
        }

        public void setEdad(int edad) {
            this.edad = edad;
        }
    }

    // Método principal
    public static void main(String[] args) {

        // Crear objetos
        Persona persona1 = new Persona("Carlos", 25);
        Persona persona2 = new Persona("María", 30);

        // Mostrar datos
        persona1.mostrarDatos();
        persona2.mostrarDatos();
    }
}
```

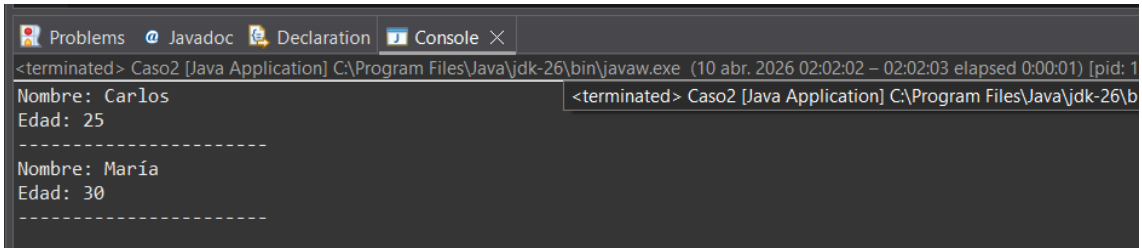



**CÓDIGO:**

```
public class Caso2 {

    // Clase Persona (definida dentro del mismo archivo)
    static class Persona {

        // Atributos
        private String nombre;
        private int edad;

        // Constructor
        public Persona(String nombre, int edad) {
            this.nombre = nombre;
            this.edad = edad;
        }

        // Método para mostrar datos
        public void mostrarDatos() {
            System.out.println("Nombre: " + nombre);
            System.out.println("Edad: " + edad);
            System.out.println("----------");
        }

        // Getters y Setters
        public String getNombre() {
            return nombre;
        }

        public void setNombre(String nombre) {
            this.nombre = nombre;
        }

        public int getEdad() {
            return edad;
        }

        public void setEdad(int edad) {
            this.edad = edad;
```

```
        }
    }

    // Método principal
    public static void main(String[] args) {

        // Crear objetos
        Persona persona1 = new Persona("Carlos", 25);
        Persona persona2 = new Persona("María", 30);

        // Mostrar datos
        persona1.mostrarDatos();
        persona2.mostrarDatos();
    }
}
```

**RESULTADO:**




## CASO 3: Sobrecarga de métodos

Consigna:

Crear una clase Operacion con métodos:
- sumar(int a, int b)
- sumar(double a, double b)
- sumar(int a, int b, int c)

Mostrar resultados desde main.

Temas:

- Sobrecarga de métodos
- Tipos de datos

# DESARROLLO:

## Clase Main.java

eclipse-workspace - Tecnicas_Programacion/src/main/java/Main.java - Eclipse IDE

File Edit Source Refactor Navigate Search Project Run Window Help

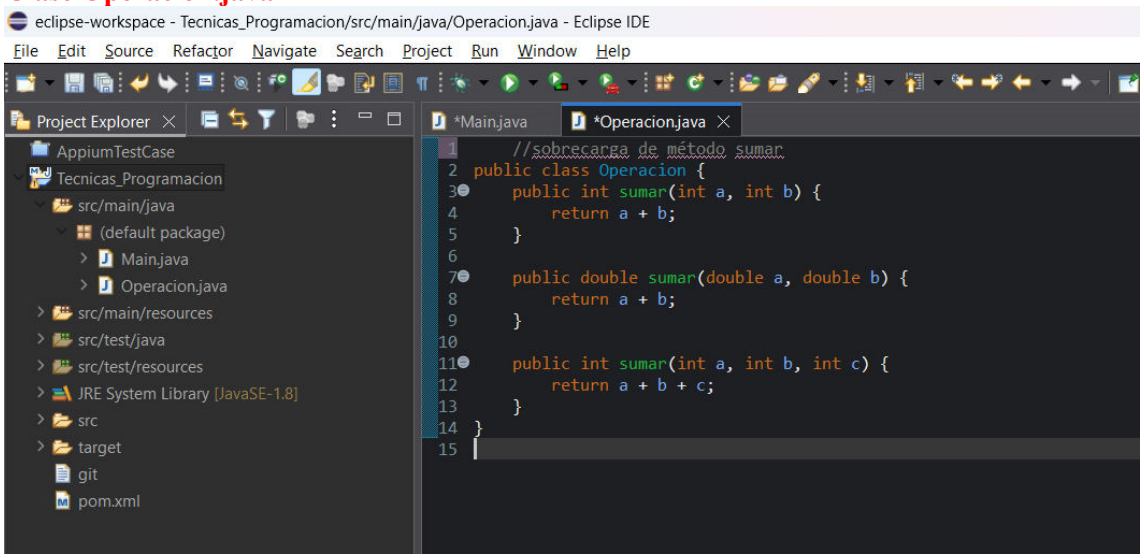
Project Explorer

- AppiumTestCase
- Tecnicas_Programacion
  - src/main/java
    - (default package)
      - Main.java
      - Operacion.java
  - src/main/resources
  - src/test/java
  - src/test/resources
  - JRE System Library [JavaSE-1.8]
  - src
  - target
  - git
  - pom.xml

*Main.java | *Operacion.java

```java
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        Operacion op = new Operacion();



        // Tipo de dato: entero
        // suma de 2 enteros
        System.out.print("Ingrese primer entero: ");
        int a = sc.nextInt();

        System.out.print("Ingrese segundo entero: ");
        int b = sc.nextInt();

        int r1 = op.sumar(a, b);
        System.out.println("Suma de 2 enteros: " + r1);
        System.out.println("-------------------------- ");

        // Tipo de dato: decimal
        // suma de 2 decimales
        System.out.print("Ingrese primer decimal: ");
        double c = sc.nextDouble();

        System.out.print("Ingrese segundo decimal: ");
        double d = sc.nextDouble();

        double r2 = op.sumar(c, d);
        System.out.println("Suma de decimales: " + r2);
        System.out.println("-------------------------- ");

        // tipo de dato: entero
        //suma de 3 enteros
        System.out.print("Ingrese primer entero: ");
        int x = sc.nextInt();

        System.out.print("Ingrese segundo entero: ");
        int y = sc.nextInt();

        System.out.print("Ingrese tercer entero: ");
        int z = sc.nextInt();

        int r3 = op.sumar(x, y, z);
        System.out.println("Suma de 3 enteros: " + r3);
        System.out.println("-------------------------- ");

        sc.close();

    }

}
```

## Clase Operación.java




## Resultado:

## CÓDIGO DEL PROGRAMA

## Main.java

Se crea la **clase Main**, donde se ingresan **datos de distintos tipos (enteros y decimales).** Además, se ejecutan los métodos de la clase Operacion para realizar las sumas y **mostrar los resultados por consola.**

```
import java.util.Scanner;

public class Main {

        public static void main(String[] args) {

                Scanner sc = new Scanner(System.in);
                Operacion op = new Operacion();


        // Tipo de dato: entero
```

```
            // suma de 2 enteros
        System.out.print("Ingrese primer entero: ");
        int a = sc.nextInt();

        System.out.print("Ingrese segundo entero: ");
        int b = sc.nextInt();

        int r1 = op.sumar(a, b);
        System.out.println("Suma de 2 enteros: " + r1);
        System.out.println("--------------------------- ");

        // Tipo de dato: decimal
        // suma de 2 decimales
        System.out.print("Ingrese primer decimal: ");
        double c = sc.nextDouble();

        System.out.print("Ingrese segundo decimal: ");
        double d = sc.nextDouble();

        double r2 = op.sumar(c, d);
        System.out.println("Suma de decimales: " + r2);
        System.out.println("--------------------------- ");

        // tipo de dato: entero
        //suma de 3 enteros
        System.out.print("Ingrese primer entero: ");
        int x = sc.nextInt();

        System.out.print("Ingrese segundo entero: ");
        int y = sc.nextInt();

        System.out.print("Ingrese tercer entero: ");
        int z = sc.nextInt();

        int r3 = op.sumar(x, y, z);
        System.out.println("Suma de 3 enteros: " + r3);
        System.out.println("--------------------------- ");

        sc.close();

        }

}
```

## Operación. Java

Se crea la clase Operacion, donde se aplica **la sobrecarga del método** sumar. Esta clase contiene varios métodos con el mismo nombre, pero con diferentes parámetros .

```
        //sobrecarga de método sumar
public class Operacion {
    public int sumar(int a, int b) {
        return a + b;
```

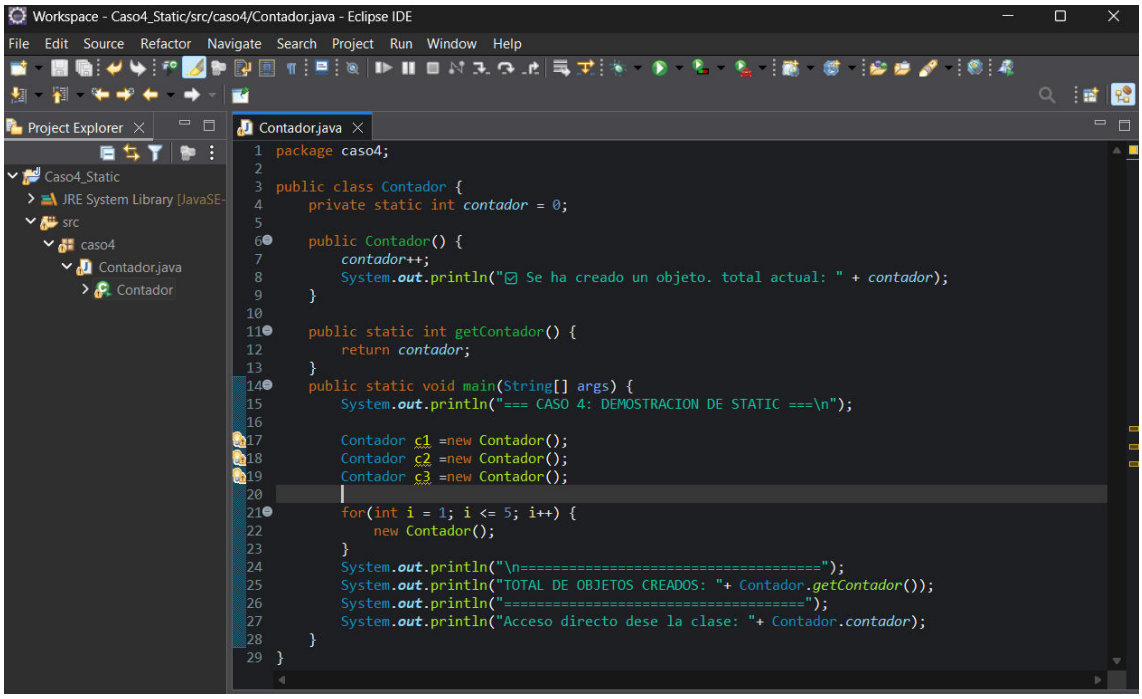
```
    }

    public double sumar(double a, double b) {
        return a + b;
    }

    public int sumar(int a, int b, int c) {
        return a + b + c;
    }
}
```

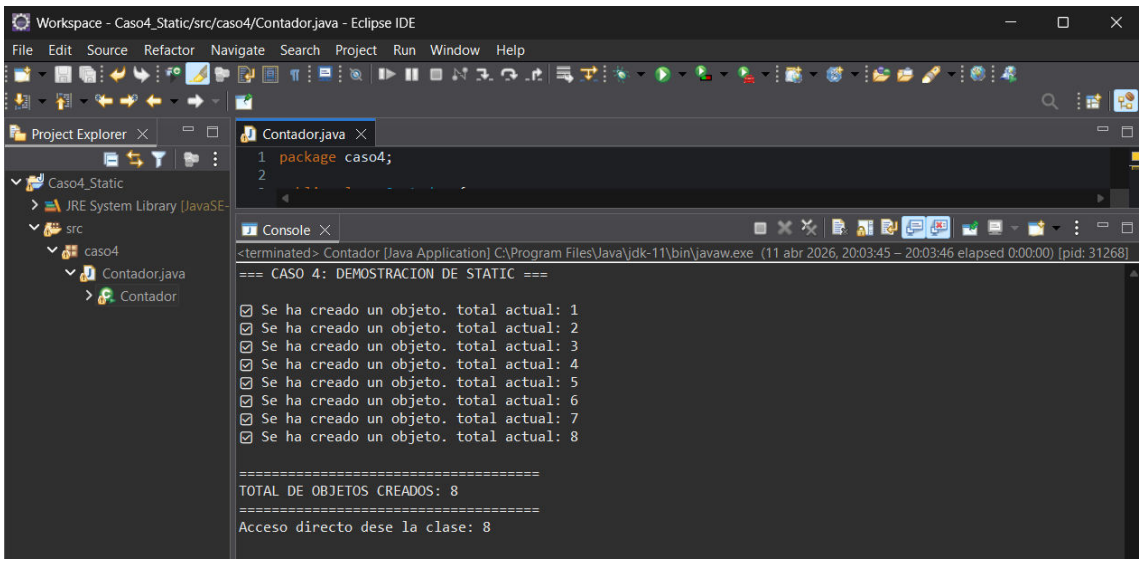
## CASO 4: Uso de static

crear una clase Contador:
- Variable estática contador
- Método que incrementa el contador cada vez que se crea un objeto

Mostrar cuántos objetos se han creado.

Temas:
- static
- Variables compartidas






## CASO 5: Lista de estudiantes

Consigna:
Crear un programa que:
- Permita ingresar nombres de estudiantes (usar ArrayList)
- Mostrar la lista completa
- Manejar error si el usuario ingresa datos inválidos

Temas:
- ArrayList
- Bucles
- Manejo básico de errores (try-catch)

## DESARROLLO

```
public class Main {
   public static void main(String[] args) {
      System.out.println("Hola mundo desde consola");
      IngresarEstudiantes();


   }

   public static void IngresarEstudiantes() {


      try {
          Scanner sc = new Scanner(System.in);
         ArrayList<String> estudiantes = new ArrayList<>();
         String nombre;
```

```
        System.out.println("REGISTRO DE ESTUDIANTES");

        // Bucle para ingresar nombres
        while (true) {
            System.out.print("Ingrese nombre del estudiante (o 'salir' para terminar): ");
            nombre = sc.nextLine();

            if (nombre.equalsIgnoreCase("salir")) {
                break;
            }

            if (!nombre.matches("[a-zA-Z]+")) {
                    throw new IllegalArgumentException("Solo se permiten letras");
            }

            estudiantes.add(nombre);
        }

        // Mostrar lista
        System.out.println("\nLista de estudiantes:");
        for (String est : estudiantes) {
            System.out.println("- " + est);
        }

        sc.close();
    }catch (IllegalArgumentException e) {
        System.out.println(e.getMessage());
    }

  }
}
```

## Preguntas de Reflexión

### ¿Cuál es la diferencia entre un método normal y uno static?

Un método normal es aquel que pertenece a un objeto, es decir, necesitas crear una instancia de la clase para poder usarlo y generalmente trabaja con los datos propios de ese objeto, como el nombre o la edad en una persona; en cambio, un método static pertenece directamente a la clase, por lo que no necesitas crear ningún objeto para ejecutarlo y se usa más para funciones generales que no dependen de información específica de una instancia.

### ¿En qué casos usarías sobrecarga de métodos?

La sobrecarga de métodos la usaría cuando quiero realizar una misma acción pero con diferentes tipos o cantidades de datos, por ejemplo, cuando un método puede recibir solo un nombre, o un nombre y una edad, sin tener que crear varios métodos con nombres distintos, lo que hace que el código sea más limpio, flexible y fácil de entender.